Inside this test case, Rails generated a single commented-out test called `"the truth"`. The `test...do` syntax may seem surprising at first, but here `ActiveSupport::TestCase` is combining a class method, optional parentheses, and a block to make defining a test method the tiniest bit simpler for you. Sometimes it's the little things that make all the difference.

The `assert` line in this method is a test. It isn't much of one, though—all it does is test that `true` is true. Clearly, this is a placeholder, one that's intended to be replaced by your actual tests.

A Real Unit Test

Let's get on to the business of testing validation. First, if we create a product with no attributes set, we'll expect it to be invalid and for an error to be associated with each field. We can use the model's `errors` and `invalid?` methods to see if it validates, and we can use the `any?` method of the error list to see if an error is associated with a particular attribute.

Now that we know *what* to test, we need to know *how* to tell the test framework whether our code passes or fails. We do that using *assertions*. An assertion is a method call that tells the framework what we expect to be true. The simplest assertion is the `assert` method, which expects its argument to be true. If it is, nothing special happens. However, if the argument to `assert` is false, the assertion fails. The framework will output a message and will stop executing the test method containing the failure. In our case, we expect that an empty `Product` model won't pass validation, so we can express that expectation by asserting that it isn't valid:

```
assert product.invalid?
```

Replace the test `the truth` with the following code:

rails80/depot_b/test/models/product_test.rb

```
test "product attributes must not be empty" do
```